

# Cruise eDNA Sampling Data Exploration And Cleaning

Ella Crotty

## Contents

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>Setup</b>                                        | <b>1</b>  |
| <b>Quick cleanup of longitude</b>                   | <b>2</b>  |
| <b>Add summary columns</b>                          | <b>2</b>  |
| Marine mammal detections . . . . .                  | 2         |
| Sample stats . . . . .                              | 2         |
| <b>Sense Check Plots</b>                            | <b>2</b>  |
| Maps . . . . .                                      | 3         |
| Tips & common problems found in this map: . . . . . | 3         |
| <b>PDF ENABLE</b>                                   | <b>4</b>  |
| Distances and Sites . . . . .                       | 4         |
| Bar plots . . . . .                                 | 10        |
| Time plots . . . . .                                | 22        |
| <b>RUN THIS Warning</b>                             | <b>30</b> |

## Setup

```
# Warnings and startup messages suppressed
library(tidyverse)
library(patchwork)
library(scales)
library(ggrepel)
library(OSMscale)
library(here)
library(stringr)
library(maps)
library(ggmap)
library(plotly) # allows for plot inspection
library(geosphere) # for calculating distance between points

# Import
df_1 <- read_csv(here("SH2607_sampling_log_v7.csv"), na = "NA")
head(df_1)

## # A tibble: 6 x 35
##   `Filter #` Cruise Sample_ID   BIOREP_OF `DATE (YYYYMMDD)` `GMT-` `LINE START`
##   <chr>      <chr>    <chr>      <lgl>          <dbl> <dbl>      <dbl>
## 1 F001      SH2607 SH2607_F001 NA              20260629      7      93.3
```

```
## 2 F002      SH2607 SH2607_F002 NA                20260630      7      93.3
## 3 F003      SH2607 SH2607_F003 NA                20260630      7      93.3
## 4 F004      SH2607 SH2607_F004 NA                20260630      7      93.3
## 5 F005      SH2607 SH2607_F005 NA                20260630      7      93.3
## 6 F006      SH2607 SH2607_F006 NA                20260630      7      93.3
## # i 28 more variables: `STATION START` <dbl>, STATION_ID_START <chr>,
## #   `TIME START` <time>, `LAT START` <dbl>, `LONG START` <dbl>,
## #   `LINE END` <dbl>, `STATION END` <dbl>, STATION_ID_END <chr>,
## #   DATE_END <dbl>, `TIME END` <time>, `LAT END` <dbl>, `LONG END` <dbl>,
## #   `VOLUME FILTERED (L)` <dbl>, `SAMPLE TYPE` <chr>,
## #   `SAMPLE DESCRIPTION` <chr>, INSHORE.OFFSHORE <chr>, Technician <chr>,
## #   Biological_notes <chr>, Technical_notes <chr>, Seafood_served <chr>, ...
```

## Quick cleanup of longitude

```
df_1 <- df_1 %>%
  mutate(`LONG START` = case_when(`LONG START` > 0 ~ `LONG START`*(-1),
                                   .default = `LONG START`),
         `LONG END` = case_when(`LONG END` > 0 ~ `LONG END`*(-1),
                                .default = `LONG END`),)
```

## Add summary columns

Add T/F columns for commonly sighted species in notes Ex. Use grep to find “humpback” and mutate column Megaptera novaeangliae = T. Eldridge suggests first converting all letters in notes column to lowercase and removing special characters [stackoverflow example](#) [stackoverflow 2](#) substringing replacing “ ” with na

## Marine mammal detections

This code isn’t rendered in the HTML output

## Sample stats

Calculations of summary statistics won’t render in the HTML file.

```
write_csv(df_summary, here("SR202511_eDNASampleMetadata_Clean.csv"))
```

## Sense Check Plots

```
# Make simple sample type - hopefully no longer needed
df_plot <- df_summary

### Date fix
# datetimes my detested
df_plot <- df_plot %>%
  mutate(Date = as.Date(as.character(`DATE` (YYYYMMDD)), format = "%Y%m%d"))
```

## Maps

### Tips & common problems found in this map:

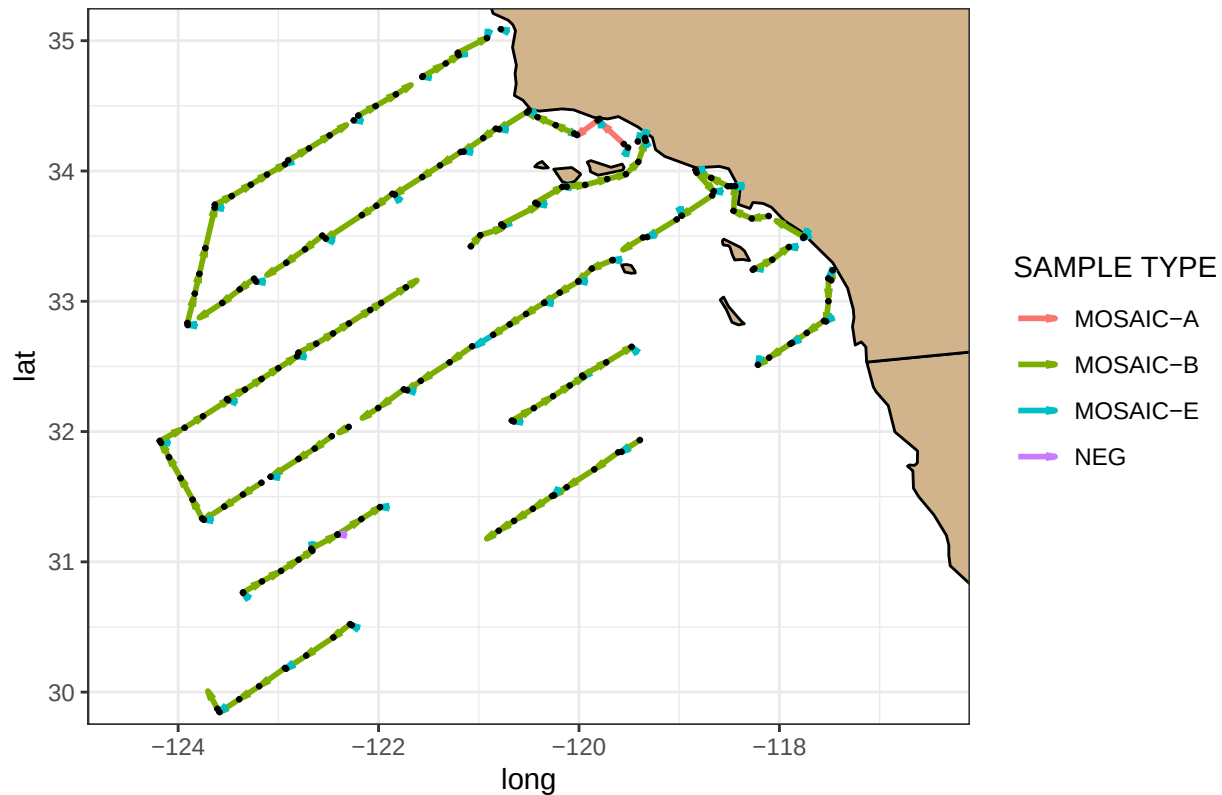
- Ella highly recommends clicking the two label flag button (sorta  $\leq$  shaped) button in the HTML plotly viewer, which is the “compare data on hover” button. This pulls up the data for CTD and on station samples which are very hard to see otherwise.
- The ggplotly() command makes a plot that you can hover your cursor over to see coordinates, which can help narrow down which samples are in the wrong place, or if transect start and end locations seem weird.
- Point in a place far from the CalCOFI grid
  - Typo: Check the written log and the coordinates of the samples immediately before and after to see if you wrote it down wrong or misread your handwriting when inputting data into the spreadsheet.
  - Google Sheets autofill: Check the coordinates immediately above: Did you intend to copy them and accidentally dragged them down (e.g. copying 33 00'00.00” instead of 32 00'00.00”)
- Transect missing
  - Typo in the sheet: Coordinate format was not correct, so it got deleted in the above degree to decimal conversion. It is usually a missing period or quotation mark.
  - To identify the missing transect use plotly zoom and “compare data on hover” buttons. Hover over the closest lines and dots to the missing transect, wiggle your cursor around, and eventually you should be able to see an NA value where there should be a coordinate. Write down the coordinate next to the NA value (if end longitude is NA, write down end latitude), search them in df\_plot, and find the filter number of your problem! Then return to the spreadsheet, find that filter number, and check its coordinates carefully.

The end of F120 was wrong - Ella copied & pasted some other station's coordinates, fixed it by referencing the coordinate photos.

The gaps in the south are because Ella was alone and sleeping. The one gap in the north is because the sea state got too rough to sample and we skipped 3 hours or so.

F186 was missing the end, start 34.5881 -121.8272 and end at 22:25 34.6606 -121.6757, taken from a photo from Nastassia with the time cut off but the location fits the grid.

```
# You'll have to press 1 to confirm making the Maps directory if you haven't already
map_df <- map_data("world")
first <- ggplot() +
  geom_polygon(data=map_df, aes(x=long, y=lat, group=group),
    fill = "tan", color = "black") +
  theme_bw() +
  coord_fixed(xlim=c(-124.5, -116.5),ylim=c(30,35), ratio = 1.3) +
  geom_segment(data=df_plot, aes(x=`LONG START`, y=`LAT START`,
    xend=`LONG END`, yend=`LAT END`,
    color = `SAMPLE TYPE`),
    arrow = arrow(angle = 10, length = unit(0.05, "inches")),
    type = "closed", size = 1) +
  geom_point(data=df_plot, aes(x=`LONG START`, y=`LAT START`), color="black",size=0.5)
first
```



```
ggsave(here("Maps", "TransectPlot.png"), width = 3000, height = 2000, units = "px")
```

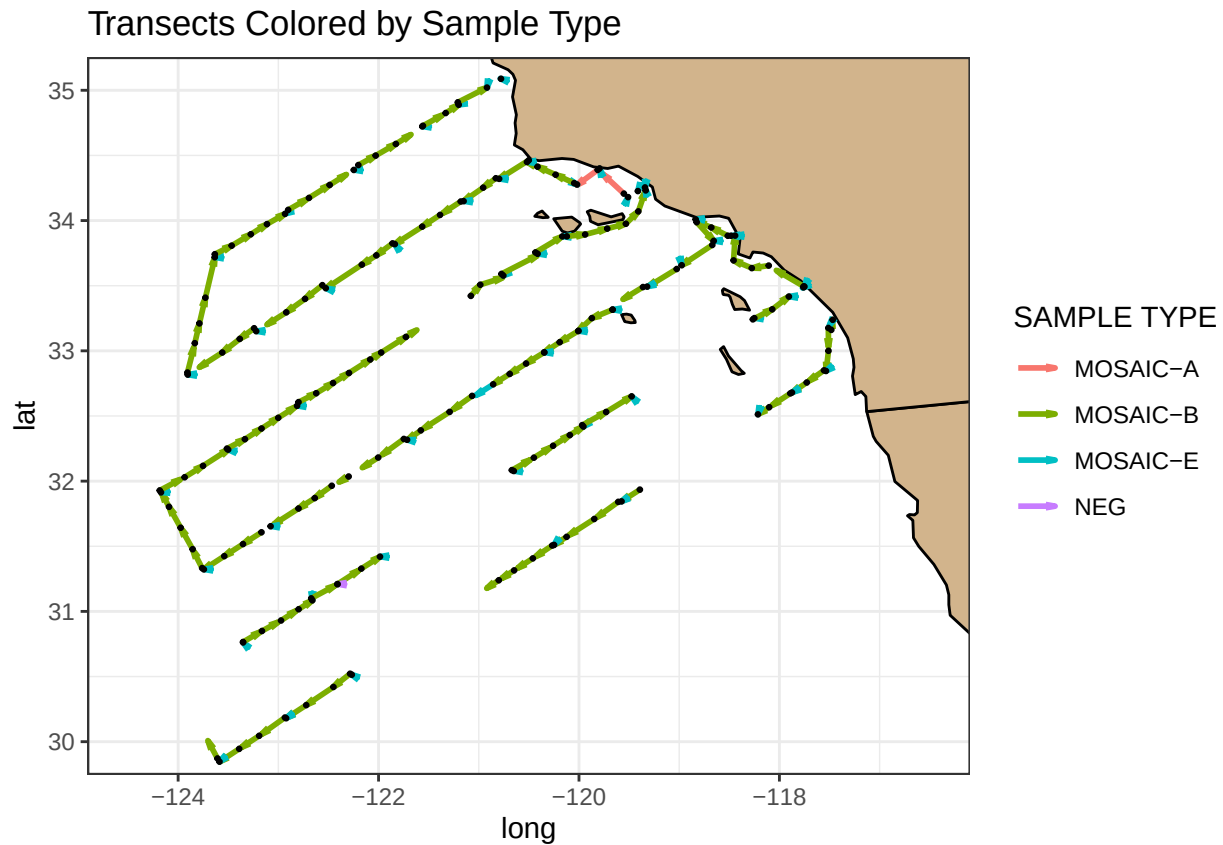
## PDF ENABLE

```
# eval = F to knit to PDF
ggplotly(first)
```

## Distances and Sites

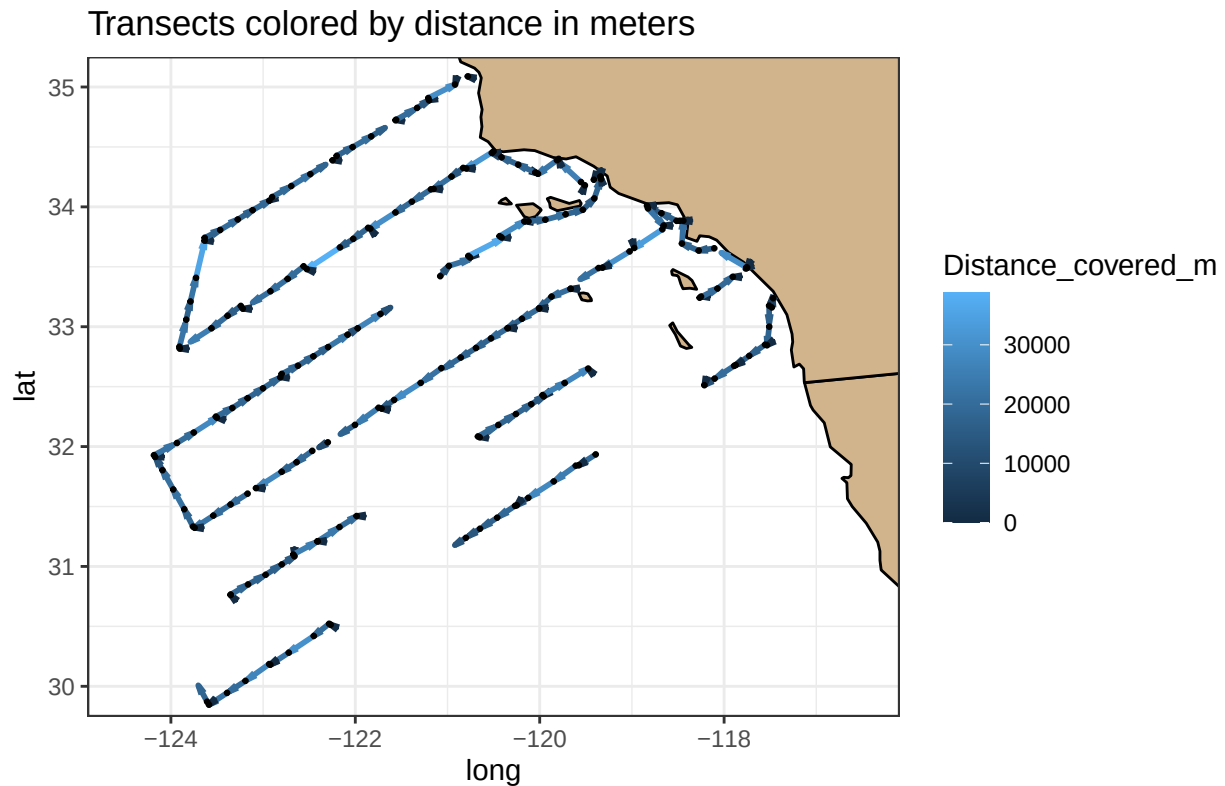
Use these maps to check for coordinate typos and make sure that the distance and site columns are working normally.

```
ggplot() +
  geom_polygon(data=map_df, aes(x=long, y=lat, group=group),
    fill = "tan", color = "black") +
  theme_bw() +
  coord_fixed(xlim=c(-124.5, -116.5),ylim=c(30,35), ratio = 1.3) +
  geom_segment(data=df_plot, aes(x=`LONG START`, y=`LAT START`,
    xend=`LONG END`, yend=`LAT END`,
    color = `SAMPLE TYPE`),
    arrow = arrow(angle = 10, length = unit(0.05, "inches")),
    type = "closed", size = 1) +
  geom_point(data=df_plot, aes(x=`LONG START`, y=`LAT START`), color="black",size=0.5) +
  ggtitle("Transects Colored by Sample Type")
```



```
ggsave(here("Maps", "TransectPlotFullSampleType.png"), width = 3000, height = 2000, units
  ↪ = "px")

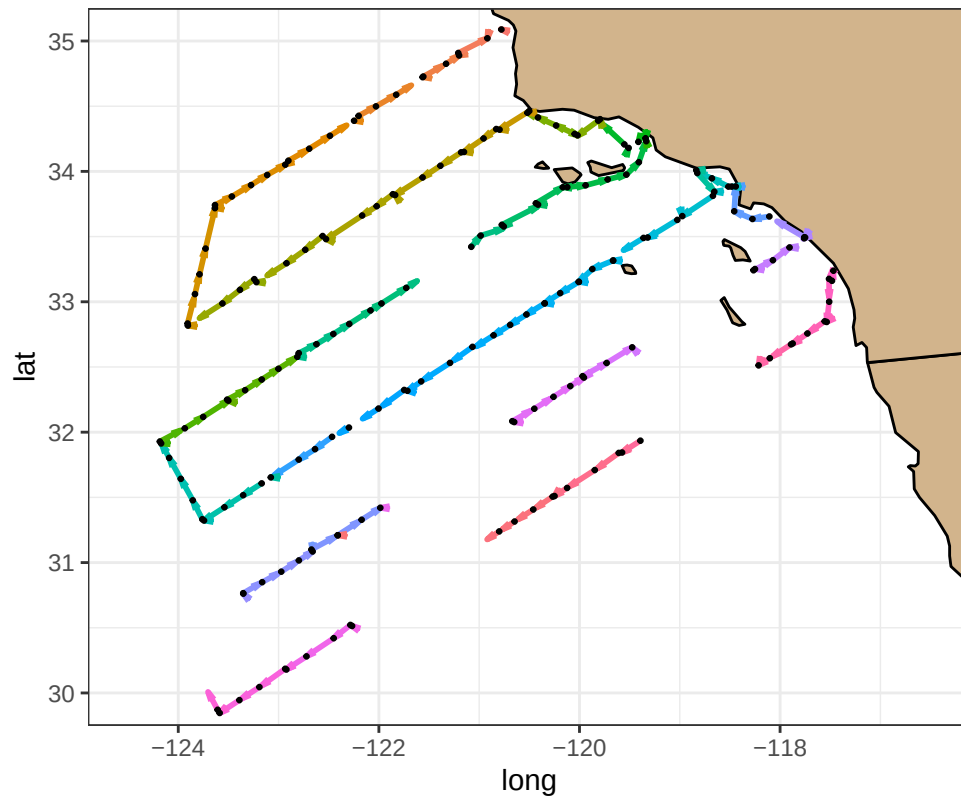
ggplot() +
  geom_polygon(data=map_df, aes(x=long, y=lat, group=group),
    fill = "tan", color = "black") +
  theme_bw() +
  coord_fixed(xlim=c(-124.5, -116.5),ylim=c(30,35), ratio = 1.3) +
  geom_segment(data=df_plot, aes(x=`LONG START`, y=`LAT START`,
    xend=`LONG END`, yend=`LAT END`,
    color = Distance_covered_m,
    arrow = arrow(angle = 10, length = unit(0.05, "inches")),
    type = "closed", size = 1) +
  geom_point(data=df_plot, aes(x=`LONG START`, y=`LAT START`), color="black",size=0.5) +
  ggtitle("Transects colored by distance in meters")
```



```
ggsave(here("Maps", "TransectPlotDistances.png"), width = 3000, height = 2000, units =
  ↪ "px")

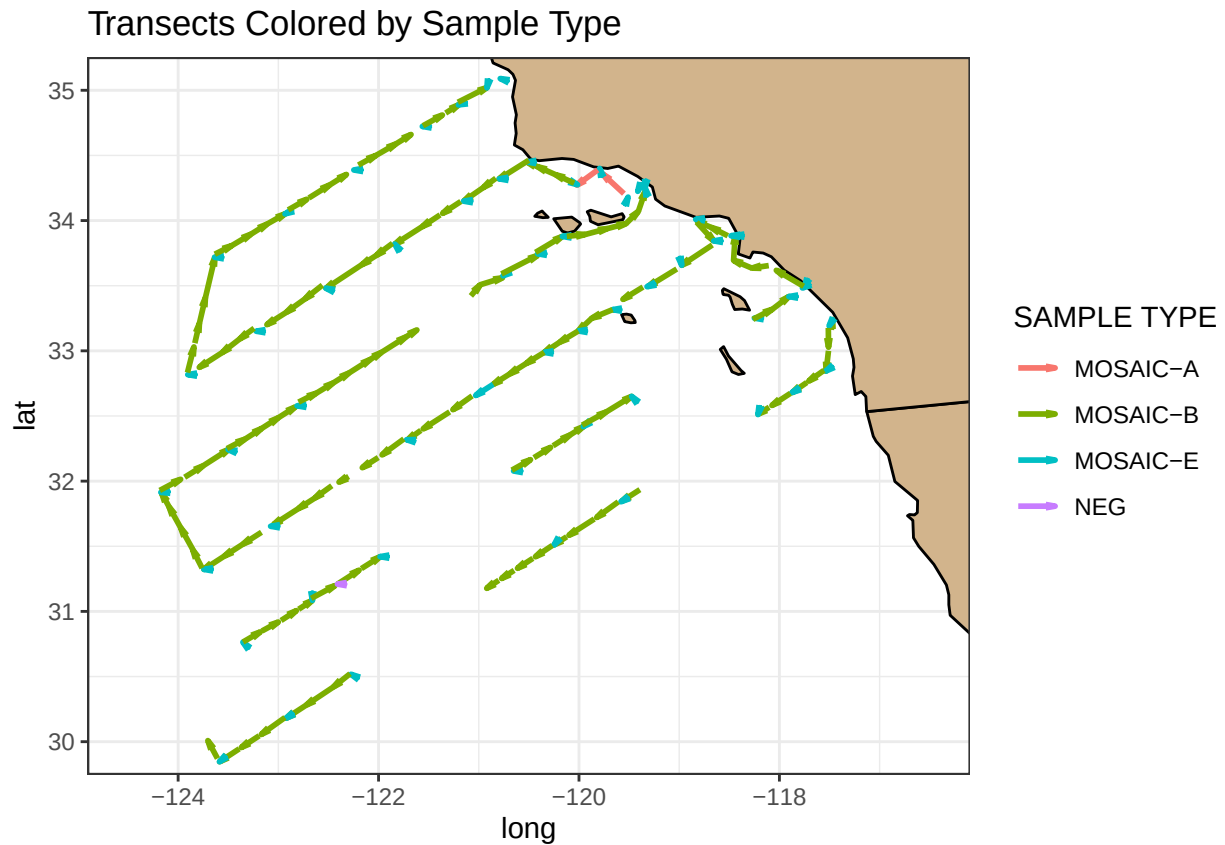
ggplot() +
  geom_polygon(data=map_df, aes(x=long, y=lat, group=group),
    fill = "tan", color = "black") +
  theme_bw() +
  theme(legend.position="none") +
  coord_fixed(xlim=c(-124.5, -116.5),ylim=c(30,35), ratio = 1.3) +
  geom_segment(data=df_plot, aes(x=`LONG START`, y=`LAT START`,
    xend=`LONG END`, yend=`LAT END`,
    color = Site),
    arrow = arrow(angle = 10, length = unit(0.05, "inches")),
    type = "closed", size = 1) +
  geom_point(data=df_plot, aes(x=`LONG START`, y=`LAT START`), color="black",size=0.5) +
  ggtitle("Transects colored by Site (start and end combo)")
```

Transects colored by Site (start and end combo)



```
ggsave(here("Maps", "TransectPlotSites.png"), width = 3000, height = 2000, units = "px")

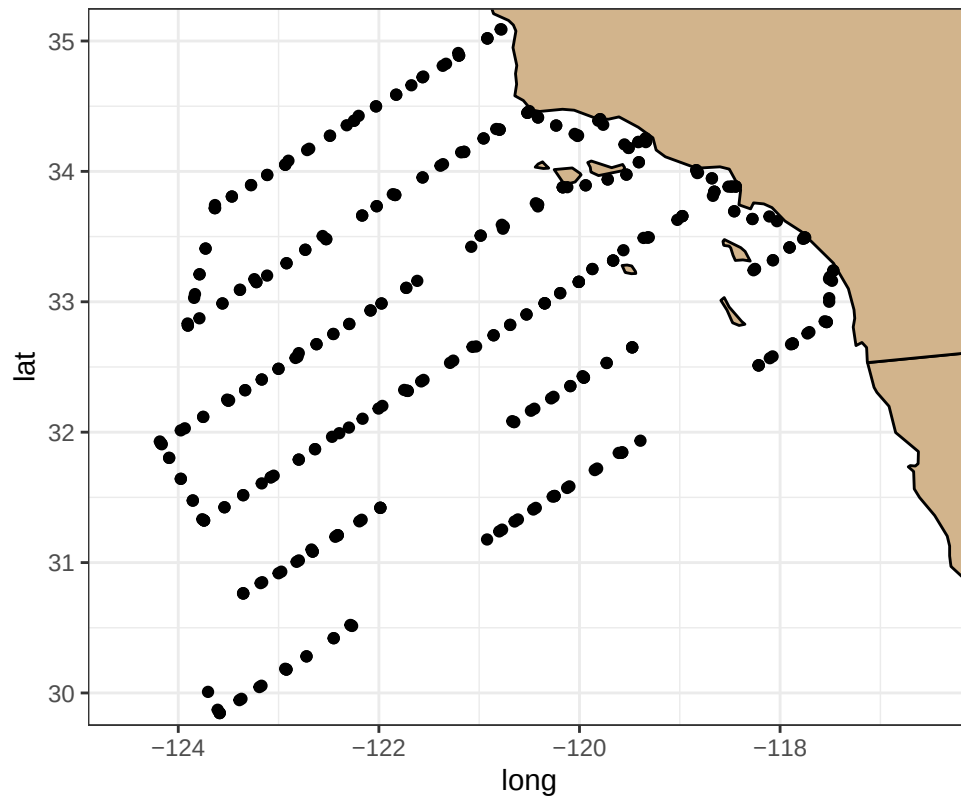
ggplot() +
  geom_polygon(data=map_df, aes(x=long, y=lat, group=group),
              fill = "tan", color = "black") +
  theme_bw() +
  coord_fixed(xlim=c(-124.5, -116.5),ylim=c(30,35), ratio = 1.3) +
  geom_segment(data=df_plot, aes(x=`LONG START`, y=`LAT START`,
                                xend=`LONG END`, yend=`LAT END`,
                                color = `SAMPLE TYPE`),
              arrow = arrow(angle = 10, length = unit(0.05, "inches")),
              type = "closed", size = 1) +
  ggtitle("Transects Colored by Sample Type")
```



```
ggsave(here("Maps", "TransectPlotNoDots.png"), width = 3000, height = 2000, units = "px")

ggplot() +
  geom_polygon(data=map_df, aes(x=long, y=lat, group=group),
    fill = "tan", color = "black") +
  theme_bw() +
  coord_fixed(xlim=c(-124.5, -116.5),ylim=c(30,35), ratio = 1.3) +
  geom_point(data = df_plot, aes(x = `LONG START`, y = `LAT START`)) +
  geom_point(data = df_plot, aes(x = `LONG END`, y = `LAT END`)) +
  ggtitle("All Transect Start And Endpoints")
```

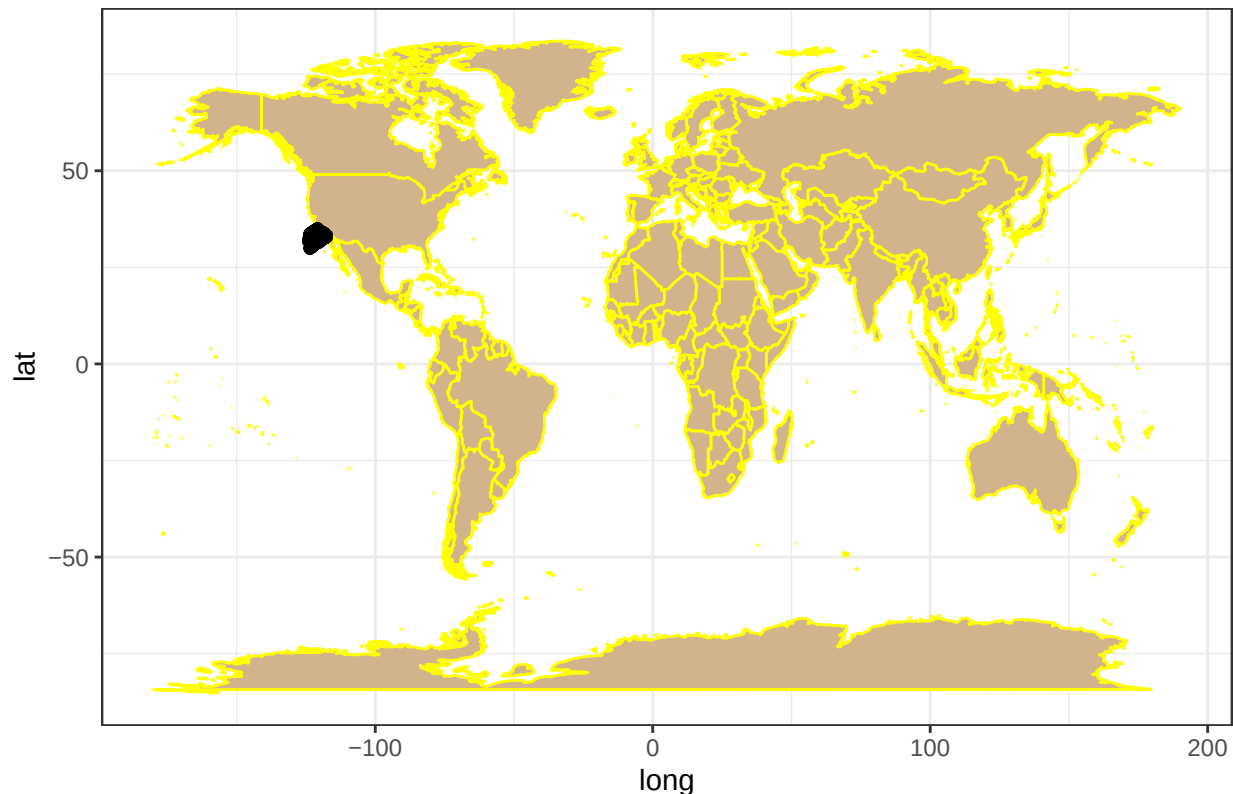
All Transect Start And Endpoints



```
ggsave(here("Maps", "DotMap.png"), width = 3000, height = 2000, units = "px")

ggplot() +
  geom_polygon(data=map_df, aes(x=long, y=lat, group=group),
    fill = "tan", color = "yellow") + # so that any misplaced black dots will
    ↪ show up really clearly
  theme_bw() +
  #coord_fixed(xlim=c(-124.5, -116.5),ylim=c(30,35), ratio = 1.3) +
  geom_point(data = df_plot, aes(x = `LONG START`, y = `LAT START`)) +
  geom_point(data = df_plot, aes(x = `LONG END`, y = `LAT END`)) +
  ggtitle("Check for coordinate typos")
```

## Check for coordinate typos



```
ggsave(here("Maps", "ZoomedOutMap.png"), width = 3000, height = 2000, units = "px")
```

## Bar plots

- BIO\_REP should mostly be 1 with a few 2 & 3
- Date, station, and line should be fairly flat distributions
- Sample type and storage box should make sense to you based on what you did - are there any NA values? Are there more partial transects than full transects? (there probably should be) etc.
- Volume histograms should show any big outliers. CTD samples are probably more clustered. There shouldn't be any 0 or NA volumes but it can be hard to see on histograms.

## Dates and Stations

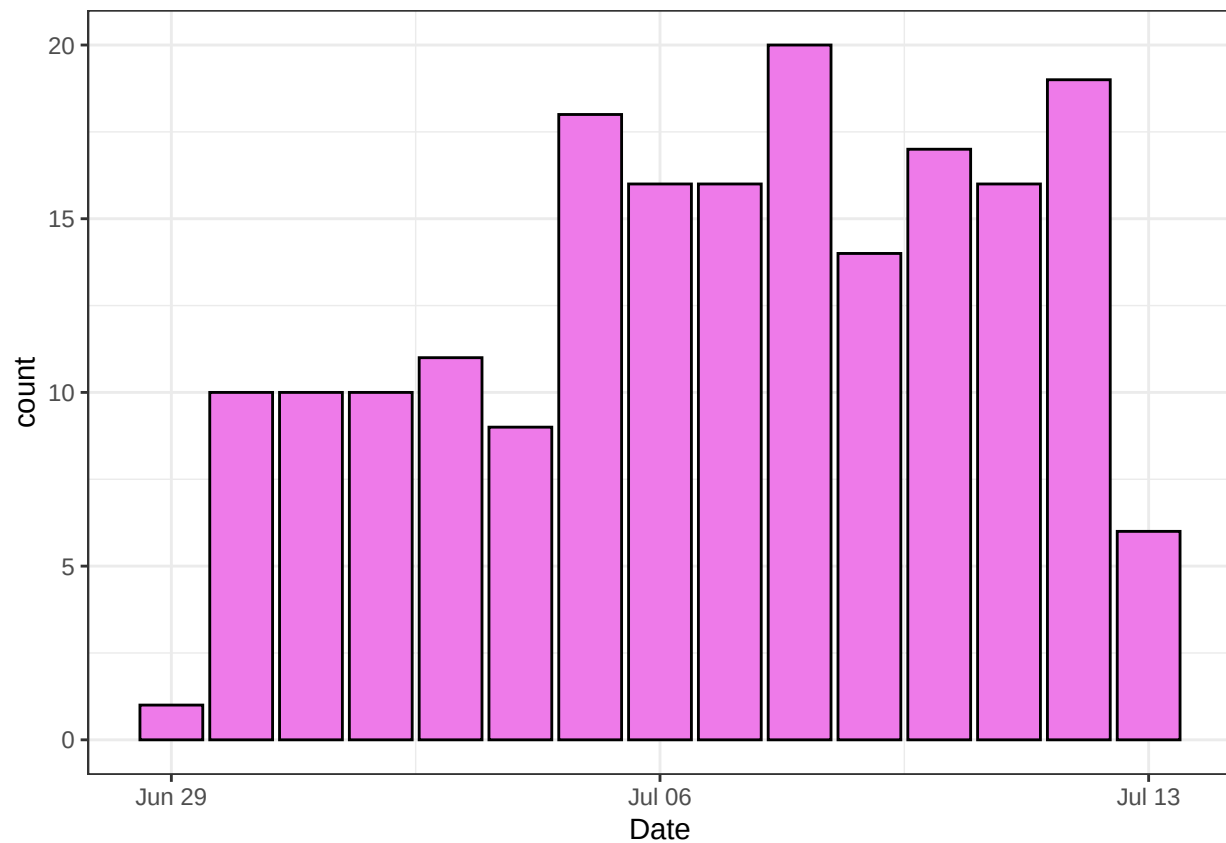
The really high count dates or stations have negatives or replicates on station.

Check anything weird against your cruise plan.

Generally, there should be more MOSAIC-B than anything else, and any CTD or MOSAIC-A >1 should be checked to make sure you intended that replicate.

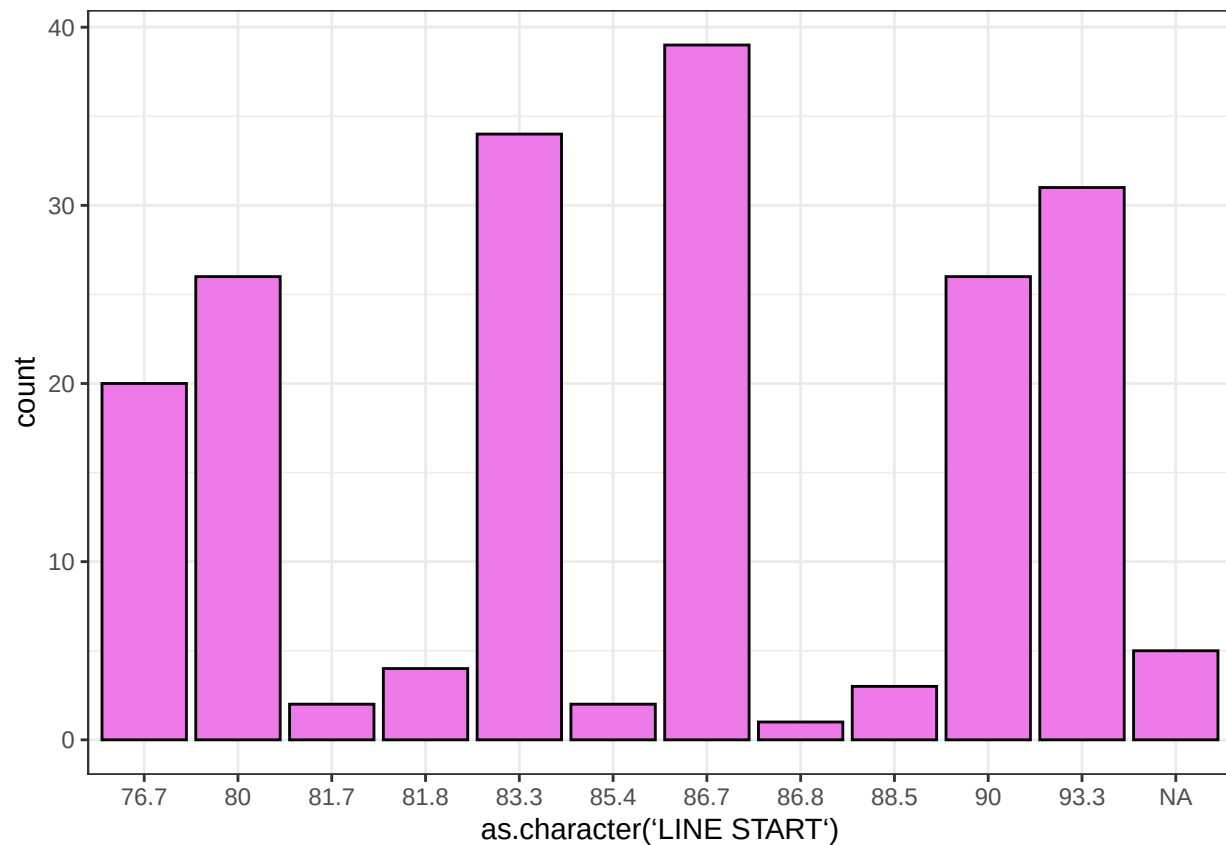
83.3-40.6 has two MOSAIC-E samples because we stopped on the wrong location first and I'd already started my sample when they realized.

```
ggplot(data = df_plot, aes(x = Date)) +  
  geom_bar(color = "black", fill = "orchid2") +  
  theme_bw()
```



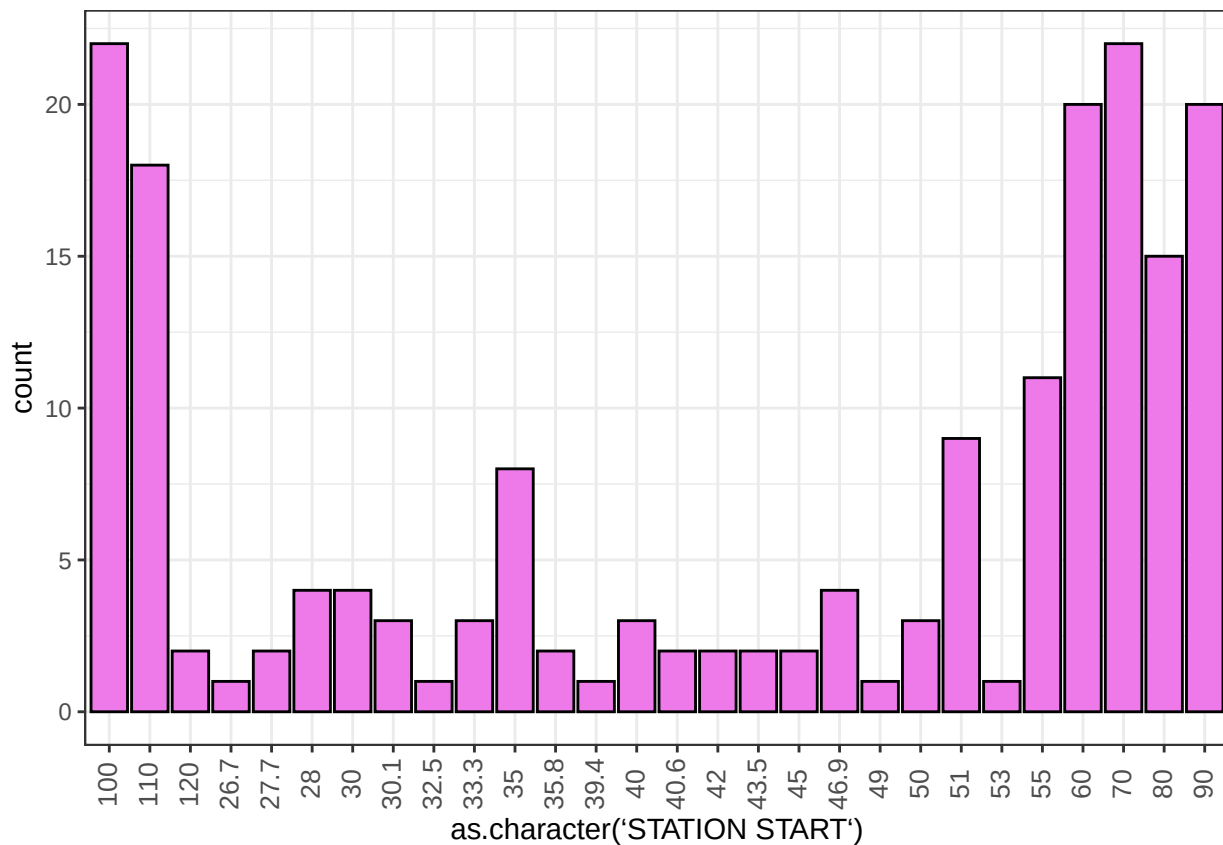
```
ggsave(filename = here("HistogramsBars", "Dates.png"), width = 2000, height = 2000, units
↪   = "px")
# Should be somewhat evenly spread

ggplot(data = df_plot, aes(x = as.character(`LINE START`))) +
  geom_bar(color = "black", fill = "orchid2") +
  theme_bw()
```



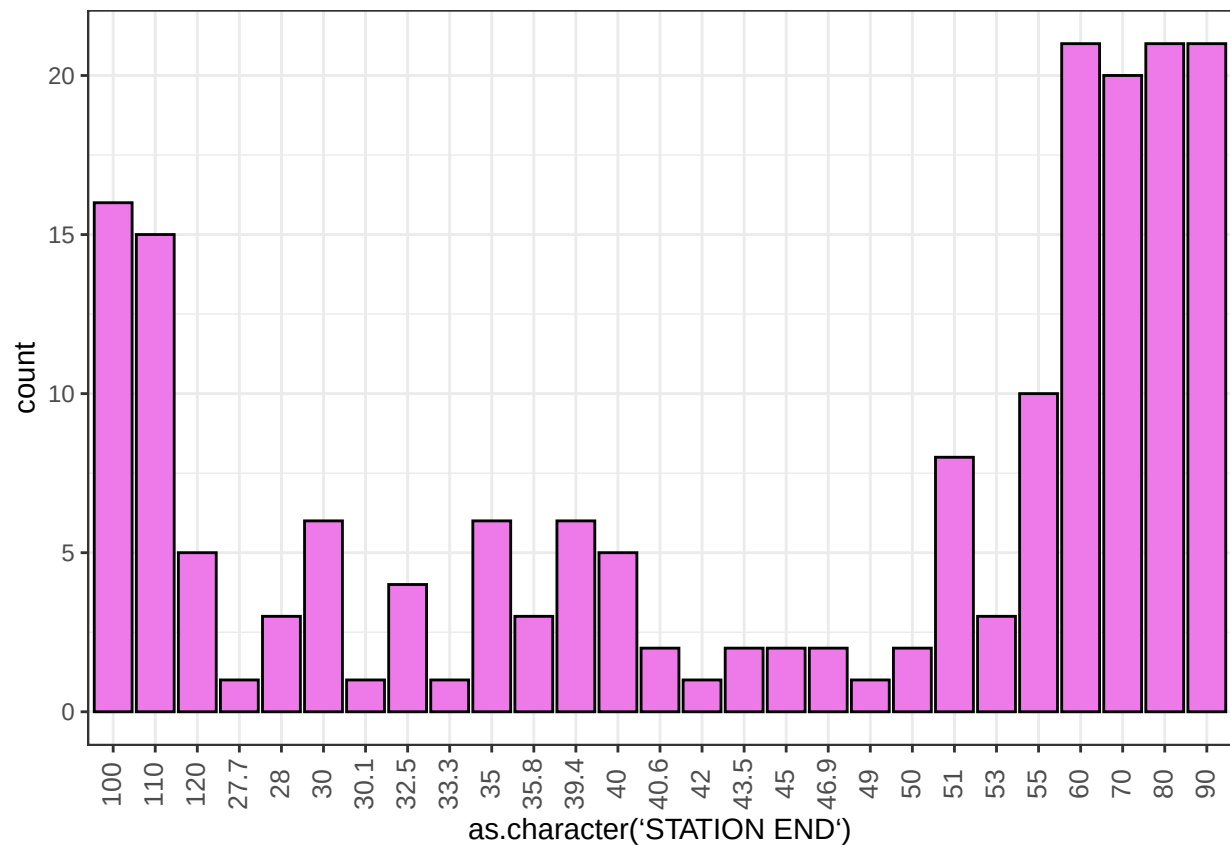
```
ggsave(filename = here("HistogramsBars", "Lines.png"), width = 2000, height = 2000, units
↪ = "px")
```

```
ggplot(data = df_plot %>% filter(!is.na(`STATION START`)),
       aes(x = as.character(`STATION START`))) +
  geom_bar(color = "black", fill = "orchid2", na.rm = T) +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.4, size = 10))
```



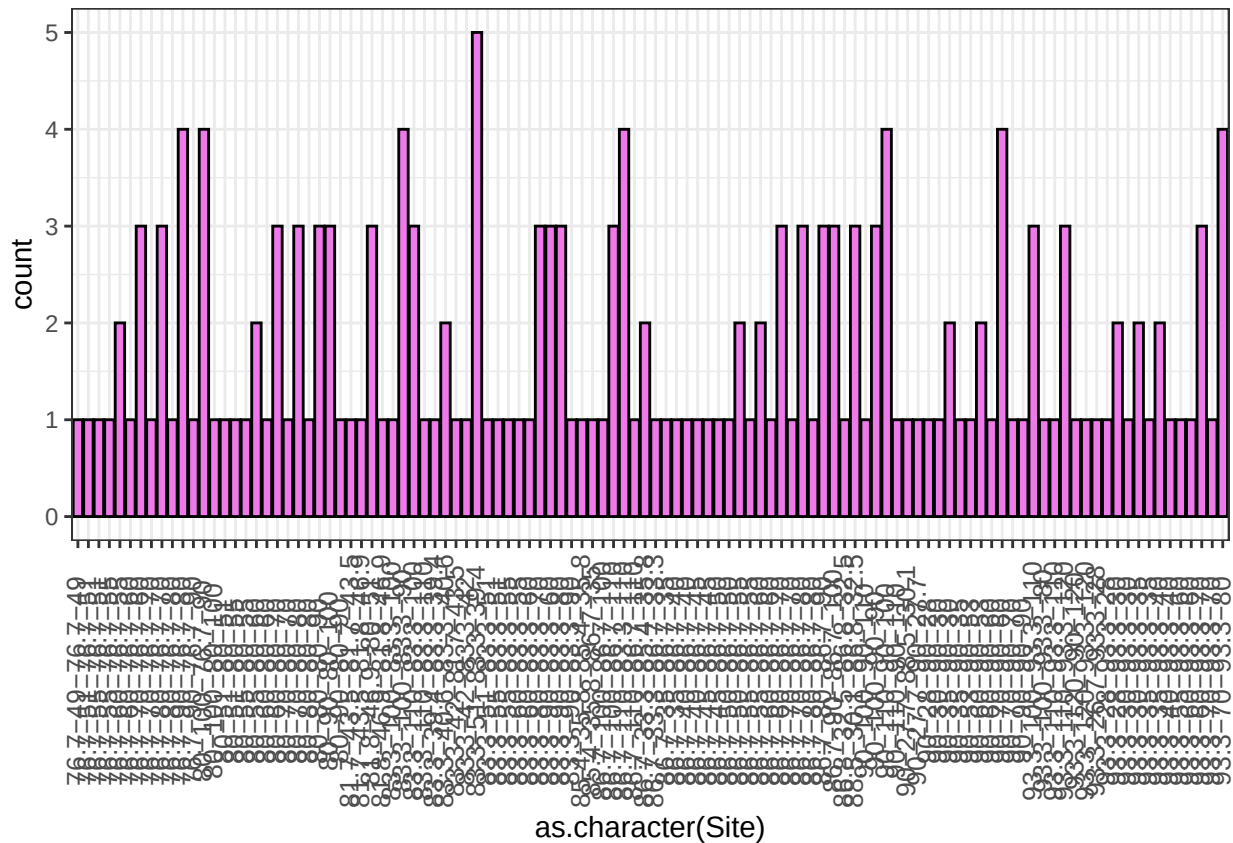
```
ggsave(filename = here("HistogramsBars", "StationStart.png"), width = 2000, height =
  ↪ 2000, units = "px")

ggplot(data = df_plot %>% filter(!is.na(`STATION END`)),
  aes(x = as.character(`STATION END`))) +
  geom_bar(color = "black", fill = "orchid2", na.rm = T) +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.4, size = 10))
```



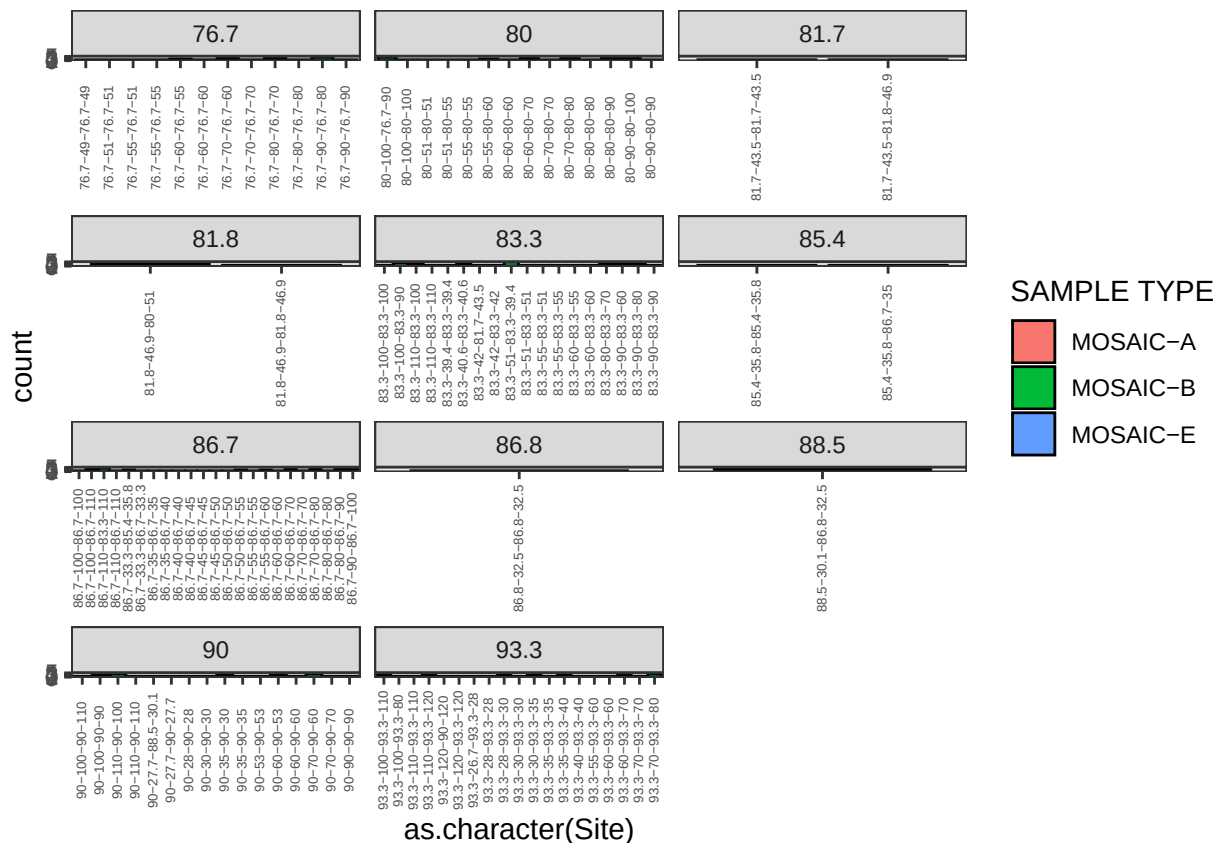
```
ggsave(filename = here("HistogramsBars", "StationEnd.png"), width = 2000, height = 2000,
  ↪ units = "px")

ggplot(data = df_plot %>% filter(!is.na(`STATION END`)),
  aes(x = as.character(Site))) +
  geom_bar(color = "black", fill = "orchid2", na.rm = T) +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.4, size = 10))
```



```
ggsave(filename = here("HistogramsBars", "Sites.png"), width = 2000, height = 2000, units
  → = "px")

ggplot(data = df_plot %>% filter(!is.na(`STATION END`)),
  aes(x = as.character(Site), fill = `SAMPLE TYPE`)) +
  geom_bar(color = "black", na.rm = T) +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.4, size = 5)) +
  facet_wrap(facets = "`LINE START`", scales = "free_x", ncol = 3)
```



```
ggsave(filename = here("HistogramsBars", "StationEndSampleType.png"), width = 2000,
  height = 2000, units = "px")
```

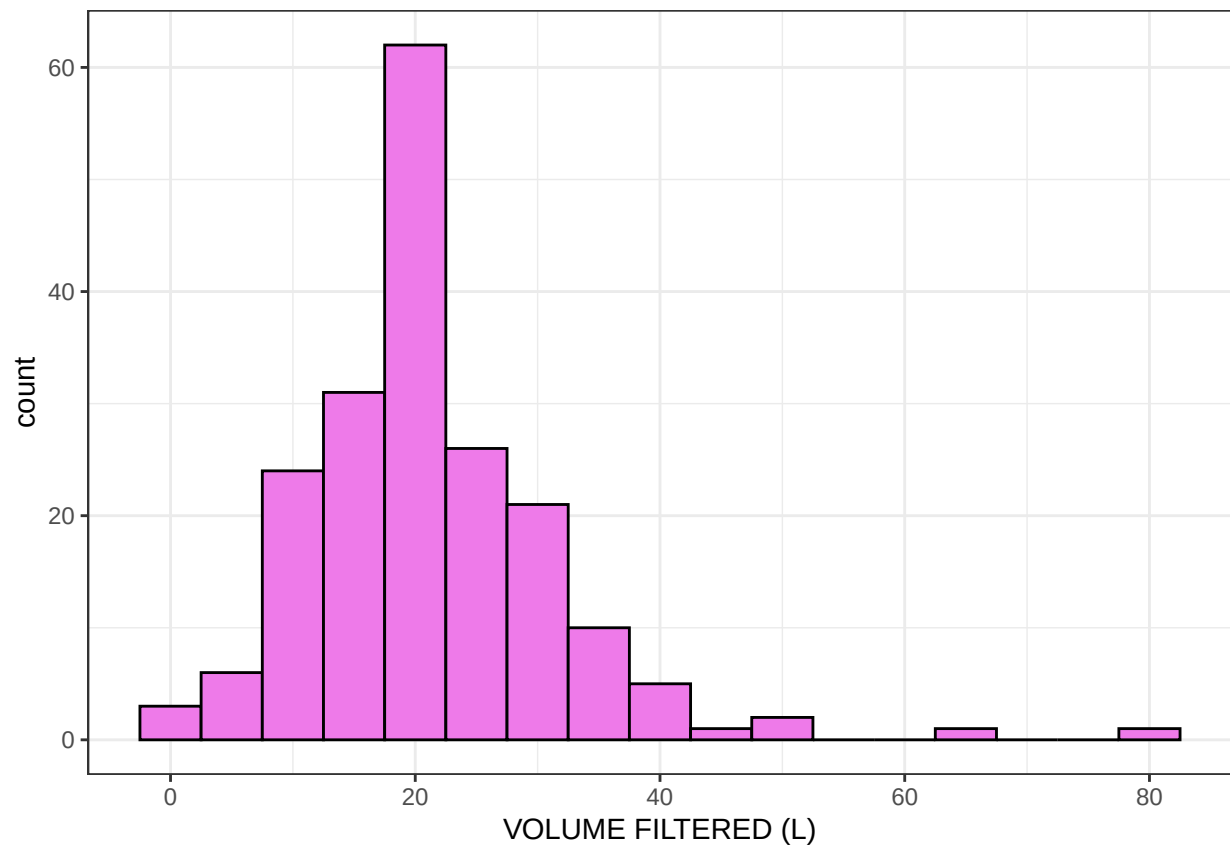
```
SiteFacetExport <- ggplot(data = df_plot %>% filter(!is.na(`STATION END`)),
  aes(x = as.character(Site), fill = `SAMPLE TYPE`)) +
  geom_bar(color = "black", na.rm = T) +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.4, size = 15)) +
  facet_wrap(facets = "`LINE START`", scales = "free_x", ncol = 2)
ggsave(filename = here("HistogramsBars", "Site_Sample_Breakdown.png"), plot =
  SiteFacetExport, width = 3000, height = 6000, units = "px")
```

## Volume Filtered

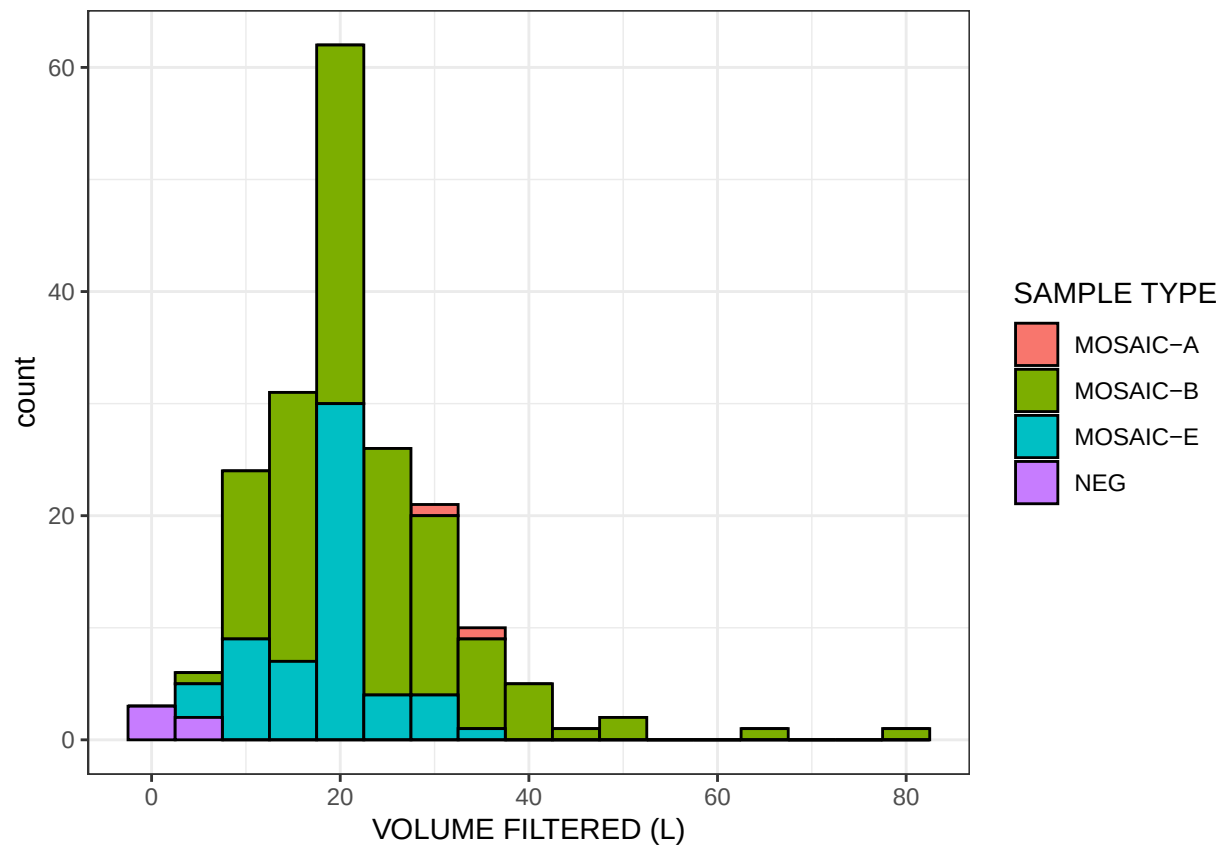
CTD samples should be clustered around one volume. Check for major outliers, and see if it matches your log.

Goal: 20 L, especially for on station. NEG should be 3-4 L.

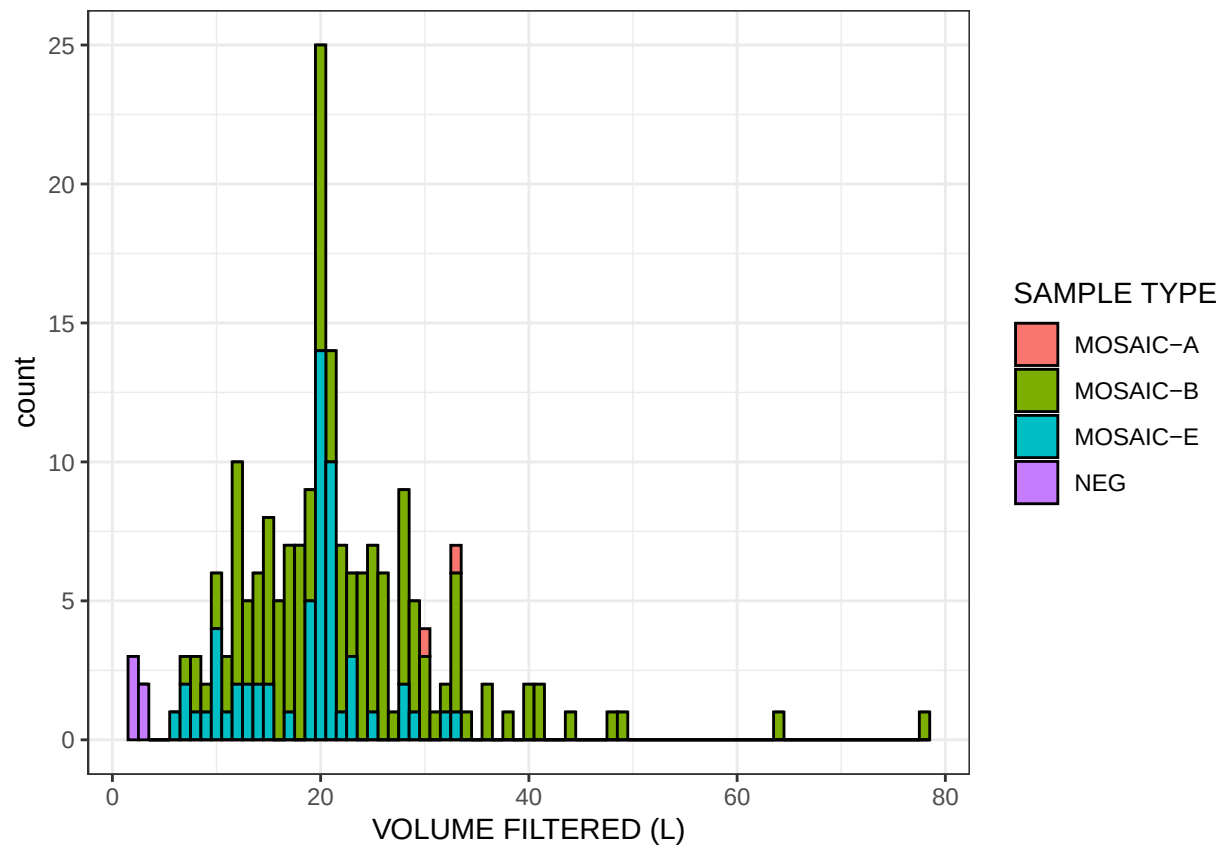
```
ggplot(data = df_plot, aes(x = `VOLUME FILTERED (L)`) +
  geom_histogram(fill = "orchid2", binwidth = 5, color = "black") +
  theme_bw())
```



```
ggplot(data = df_plot, aes(x = `VOLUME FILTERED (L)`) +  
  geom_histogram(aes(fill = `SAMPLE TYPE`), binwidth = 5, color = "black") +  
  theme_bw()
```



```
ggsave(filename = here("HistogramsBars", "VolumeTypeBig.png"), width = 3000, height =
  ↪ 6000, units = "px")
ggplot(data = df_plot, aes(x = `VOLUME FILTERED (L)`) +
  geom_histogram(aes(fill = `SAMPLE TYPE`), binwidth = 1, color = "black") +
  theme_bw())
```

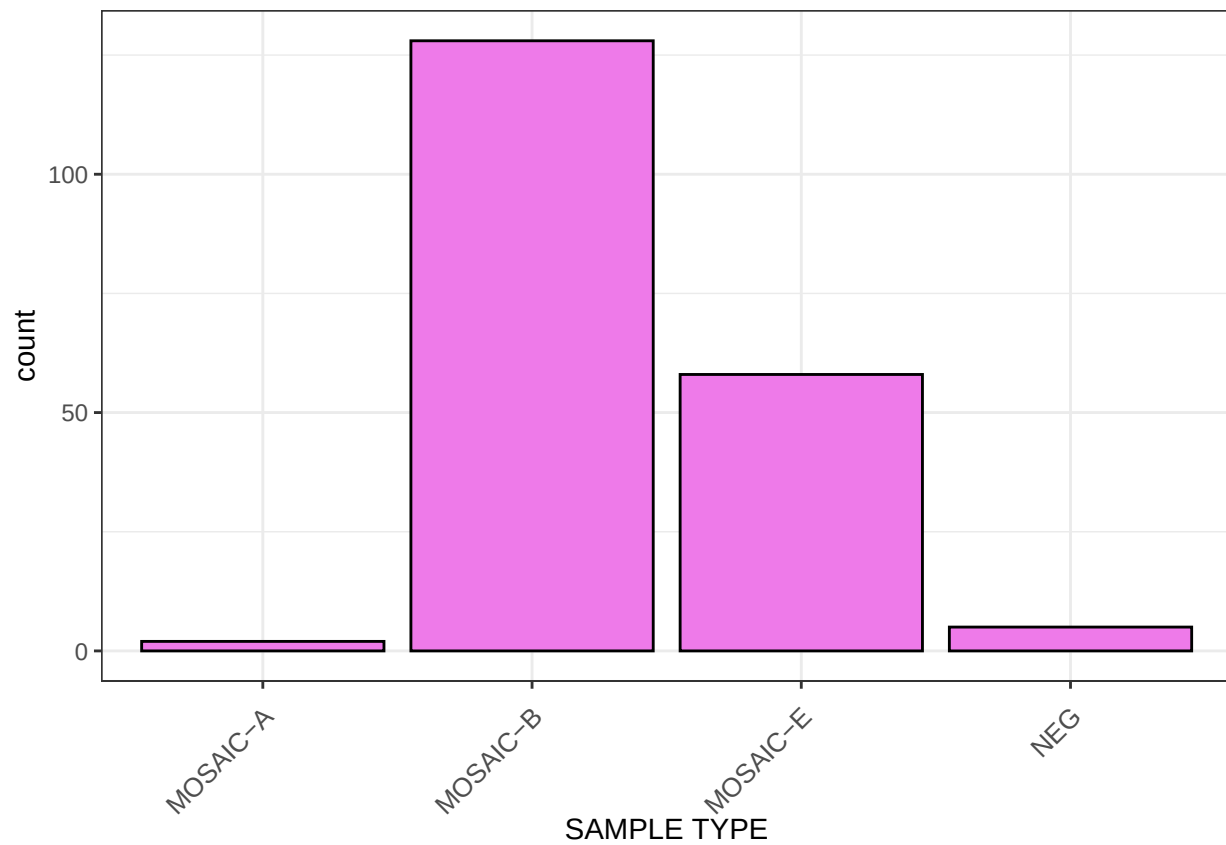


```
ggsave(filename = here("HistogramsBars", "VolumeTypeSmall.png"), width = 3000, height =
↪ 6000, units = "px")
```

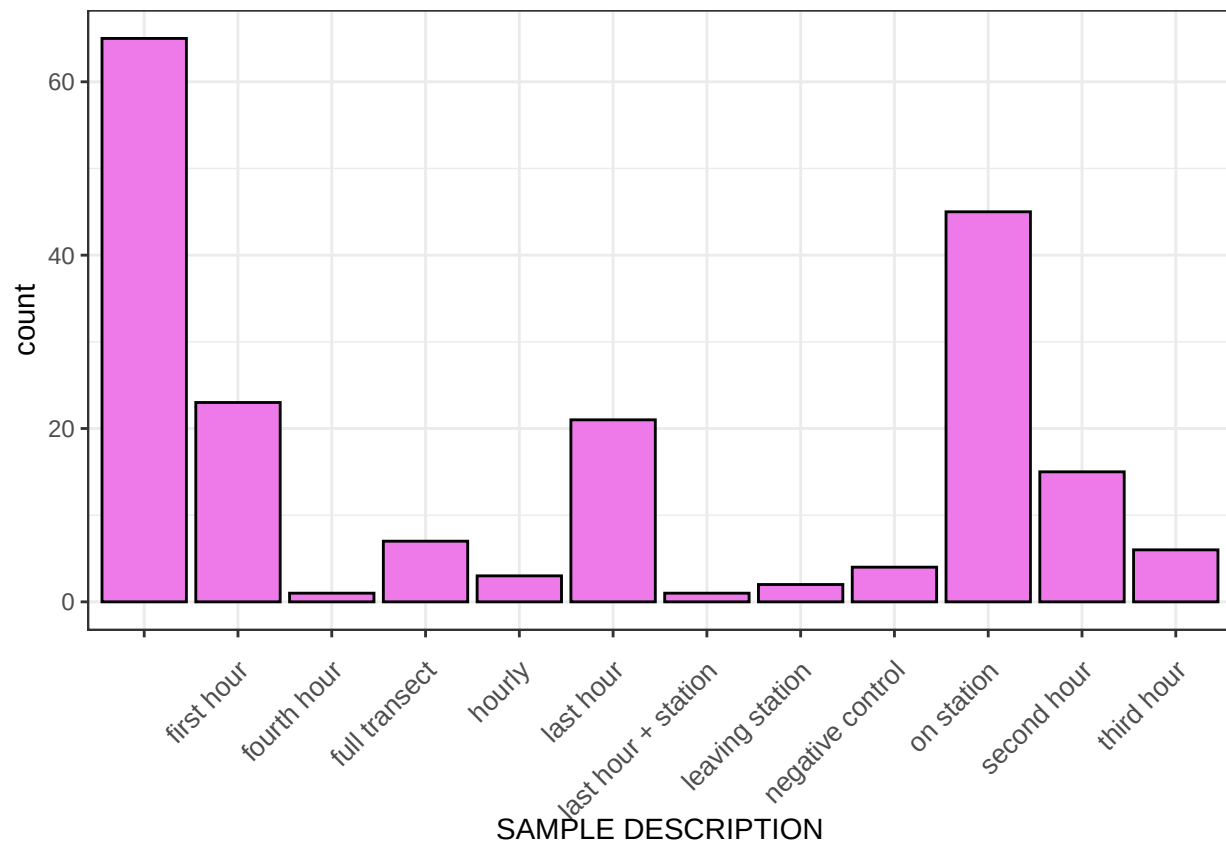
## Sample Types

This should help catch whether you've gotten all the NAs or typos in these columns

```
ggplot(data = df_plot, aes(x = `SAMPLE TYPE`)) +
  geom_bar(color = "black", fill = "orchid2") +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 45, vjust = 0.9, hjust = 1,
    size = 10))
```

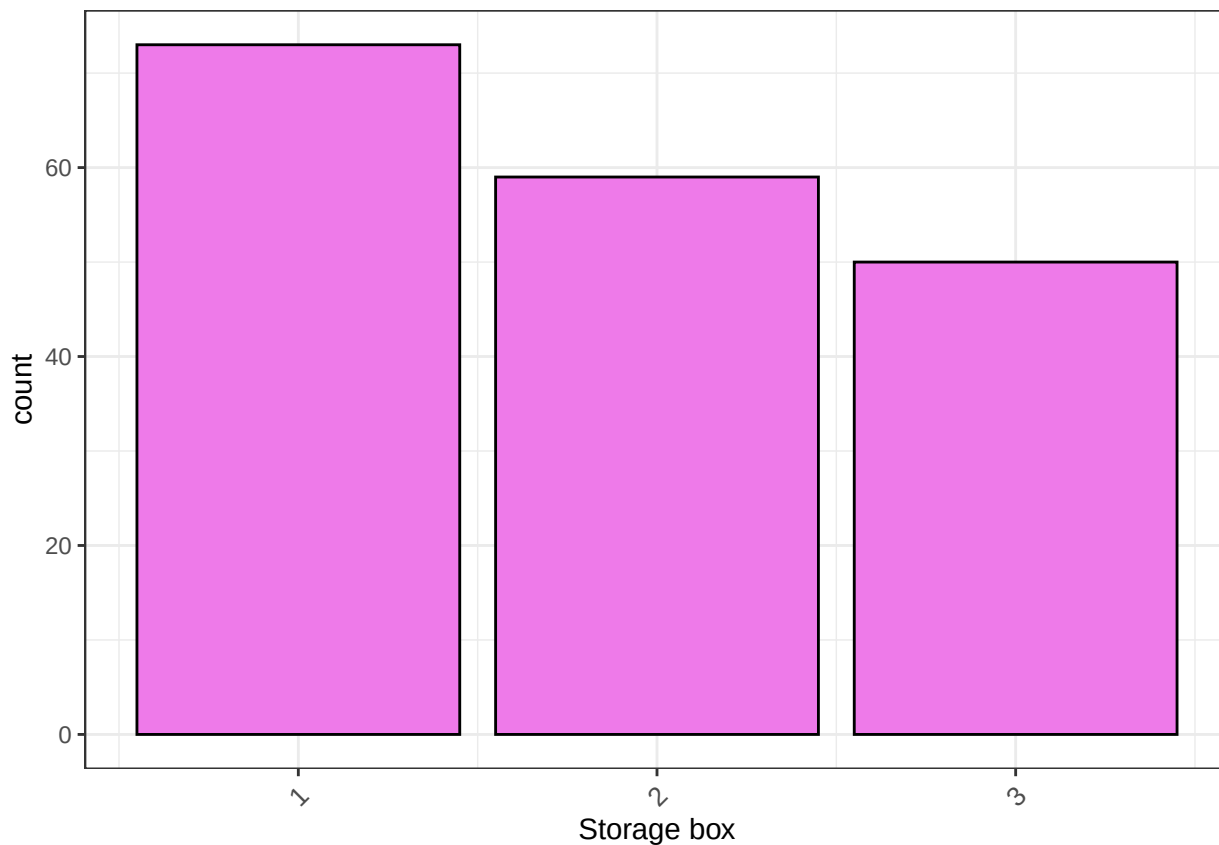


```
ggsave(filename = here("HistogramsBars", "SampleType.png"), width = 3000, height = 6000,
  ↪ units = "px")
ggplot(data = df_plot, aes(x = `SAMPLE DESCRIPTION`)) +
  geom_bar(color = "black", fill = "orchid2") +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 45, vjust = 0.9, hjust = 1,
    size = 10))
```



```
ggsave(filename = here("HistogramsBars", "SampleDescription.png"), width = 3000, height =
↳ 6000, units = "px")

ggplot(data = df_plot, aes(x = `Storage box`)) +
  geom_bar(color = "black", fill = "orchid2") +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 45, vjust = 0.9, hjust = 1,
    size = 10))
```

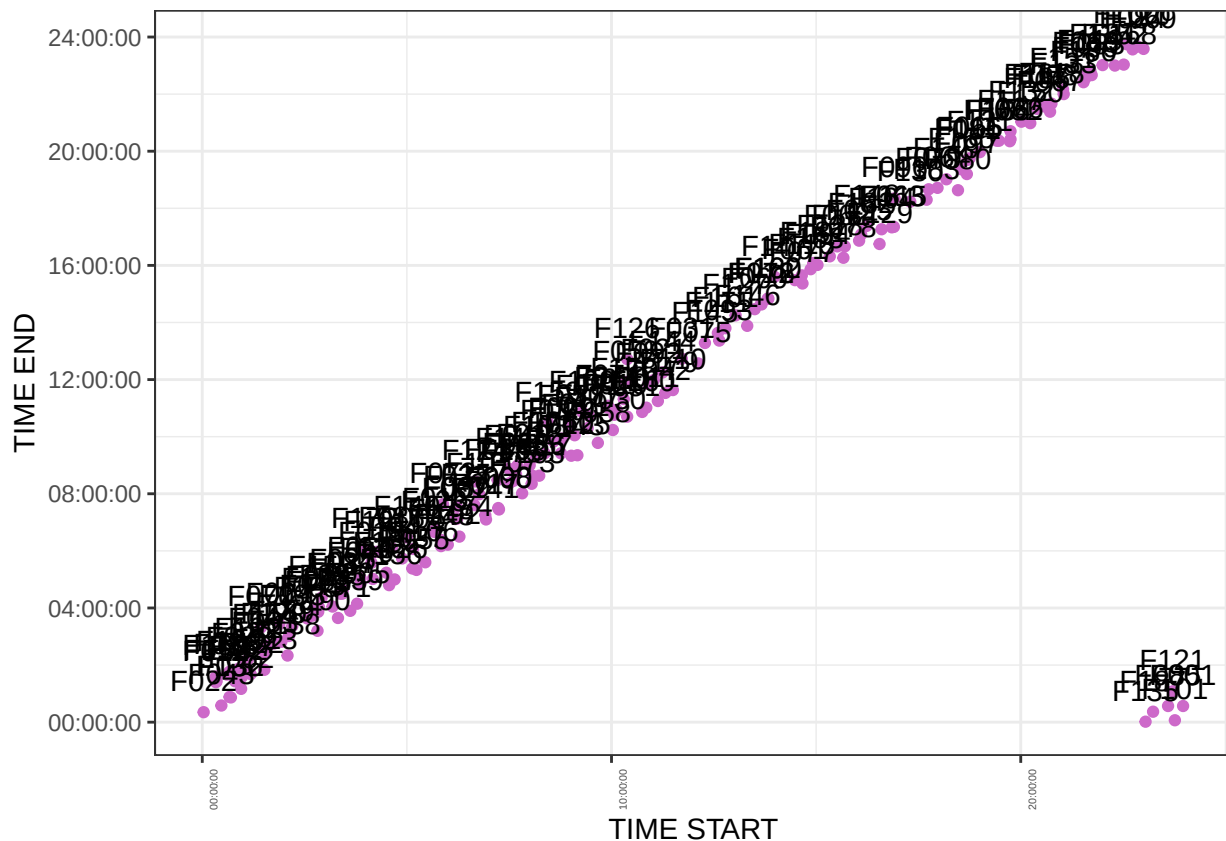


```
ggsave(filename = here("HistogramsBars", "Storage.png"), width = 3000, height = 6000,
  ↪ units = "px")
```

## Time plots

- If this plot won't render, make sure TIME.START is all times and has no characters.
- The number labels will allow you to spot-check for typos.
- Time start vs. time end should be a pretty straight line, investigate any points that are off the line based on their labels. There may be a cluster in the lower right, which is usually correctly labeled samples that started late at night and ended after midnight (e.g. technically the next morning).

```
times <- ggplot(data = df_plot, aes(x = `TIME START`, y = `TIME END`)) +
  geom_point(color = "orchid3") +
  geom_text(aes(label = `Filter #`, vjust = -1)) +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 90, vjust = 2, size = 4))
times
```



```
ggsave(filename = here("Time", "TimeDiagonal.png"), width = 2000, height = 2000, units =
  ↪ "px")
```

```
# TIME.START had one "CTD" value and wasn't parsing as hms, fixed in spreadsheet
# Unusual lengths: F025,F053,F086,F087,F109,F110(started 23xx, ended 00xx)
# F055,F135 (10 or 11 pm mislabeled as AM, fixed in spreadsheet )
# F078 (that one day we had an actual 8 hour transect)
```

Crossing days: F087, F121, F001, F101, F085, F187, F135

F087 actually had some weird spreadsheet formatting issue

```
ggplotly(times)
```

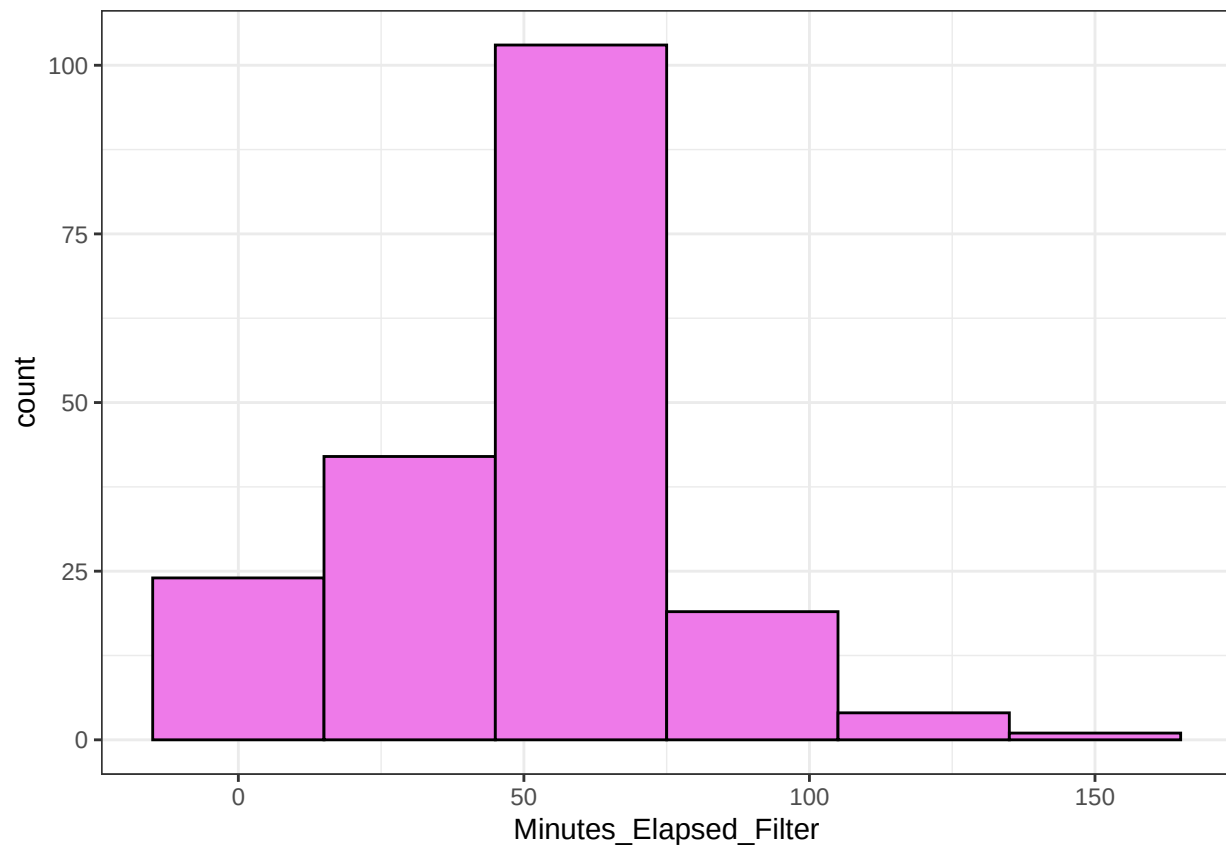
## Minutes elapsed

If there are a bunch of negative times, make sure you've got DATE\_END accurate. Try opening df\_plot and sorting Minutes\_Elapsed\_Filter least to greatest, and the problems will reveal themselves. Then go back to the spreadsheet and fix it.

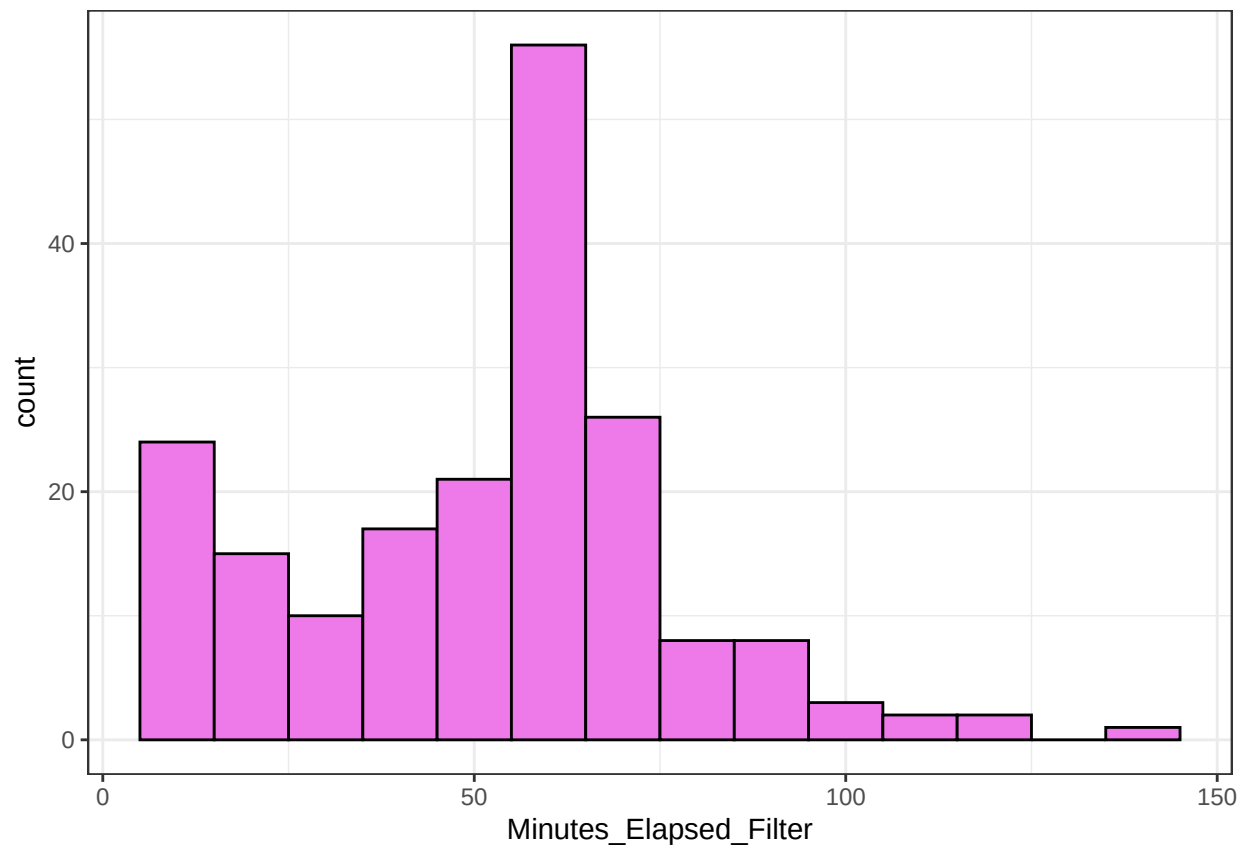
Does this seem accurate to what you did?

Should be mostly 60, with a peak at 20 minutes for on station and nothing above ~120.

```
ggplot(data = df_plot, aes(x = `Minutes_Elapsed_Filter`)) +
  geom_histogram(fill = "orchid2", color = "black", binwidth = 30) +
  theme_bw()
```

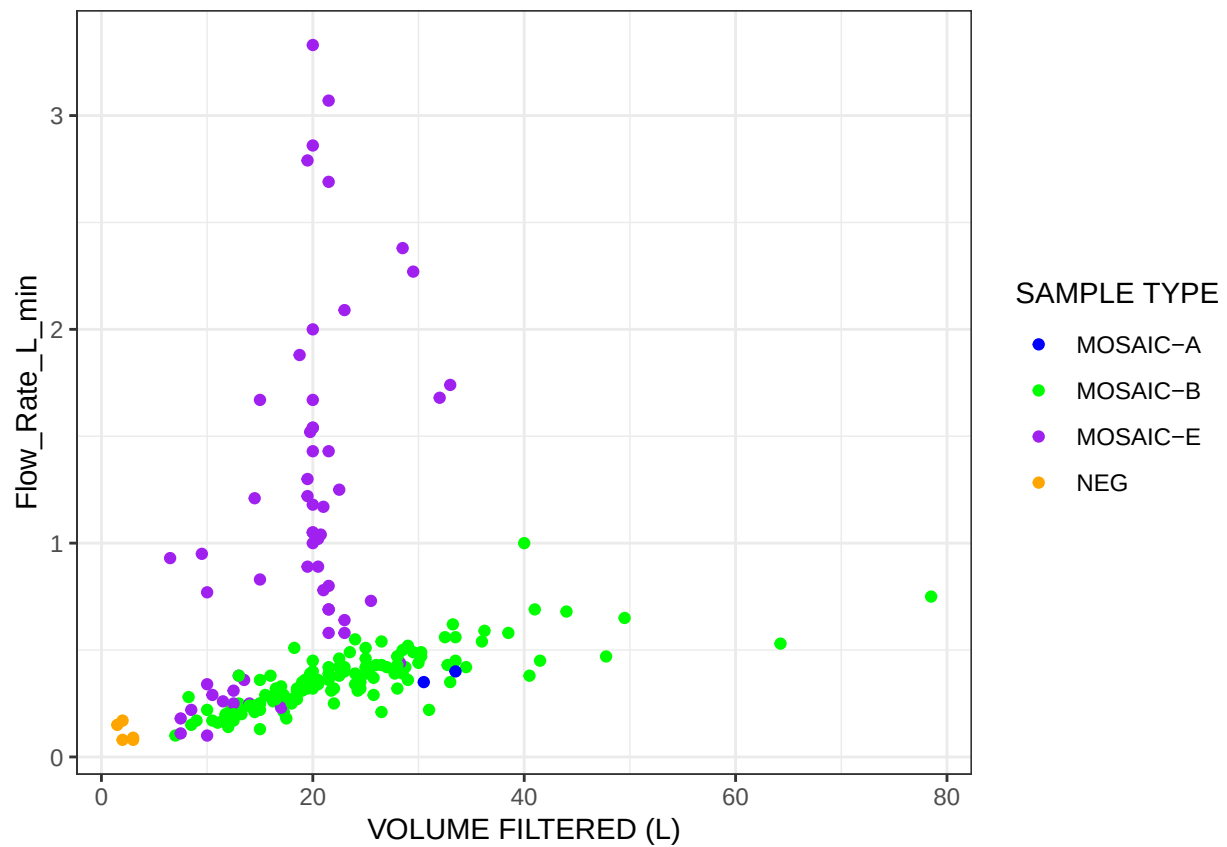


```
ggsave(filename = here("HistogramsBars", "TimeHistogram.png"), width = 2000, height =  
  ↪ 2000, units = "px")  
  
ggplot(data = df_plot, aes(x = `Minutes_Elapsed_Filter`)) +  
  geom_histogram(fill = "orchid2", color = "black", binwidth = 10) +  
  theme_bw()
```



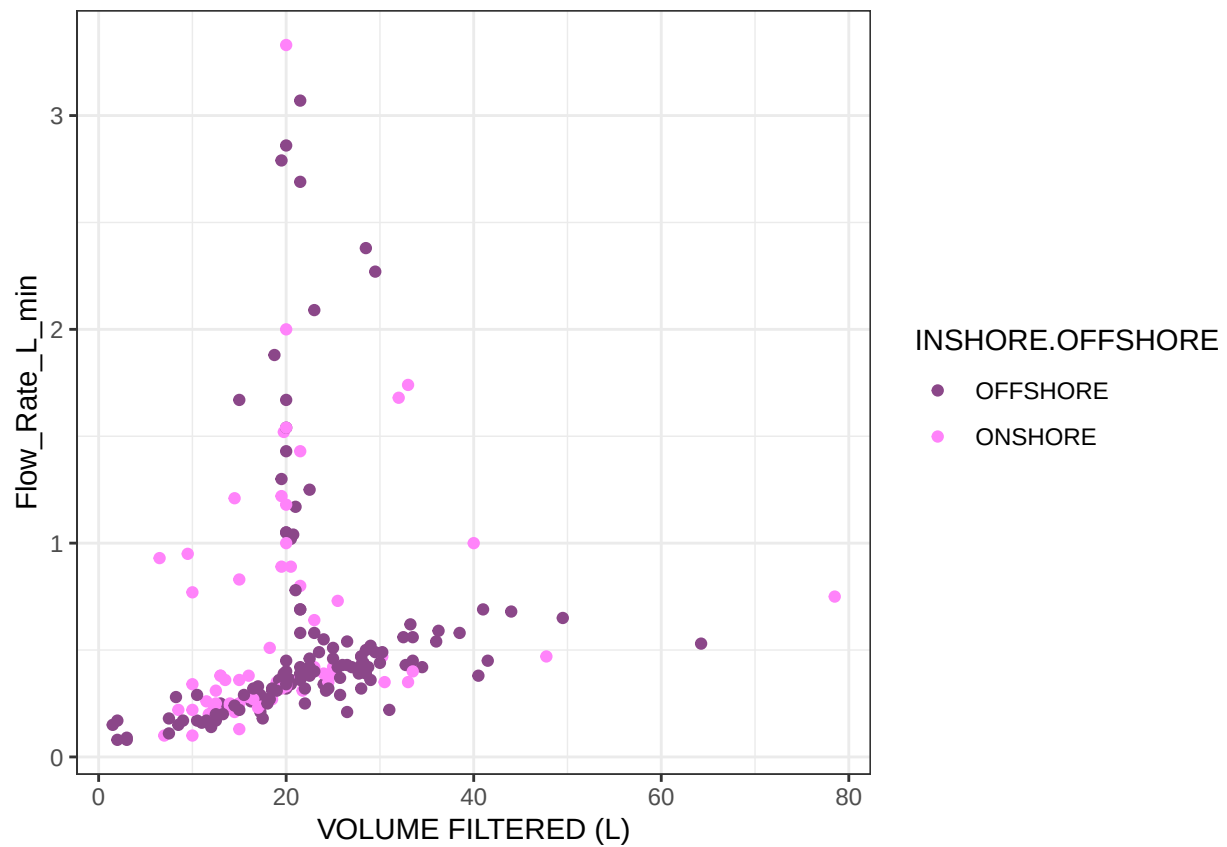
```
ggsave(filename = here("HistogramsBars", "TimeHistogram.png"), width = 2000, height =
↪ 2000, units = "px")
```

```
ggplot(df_plot, aes(x = `VOLUME FILTERED (L)`, y = Flow_Rate_L_min, color = `SAMPLE
↪ TYPE`)) +
  geom_point() +
  scale_color_manual(values = c("blue", "green", "purple", "orange")) +
  theme_bw()
```



```
ggsave(filename = here("Time", "VolumeVsFlow.png"), width = 2000, height = 2000, units =
  ↪ "px")

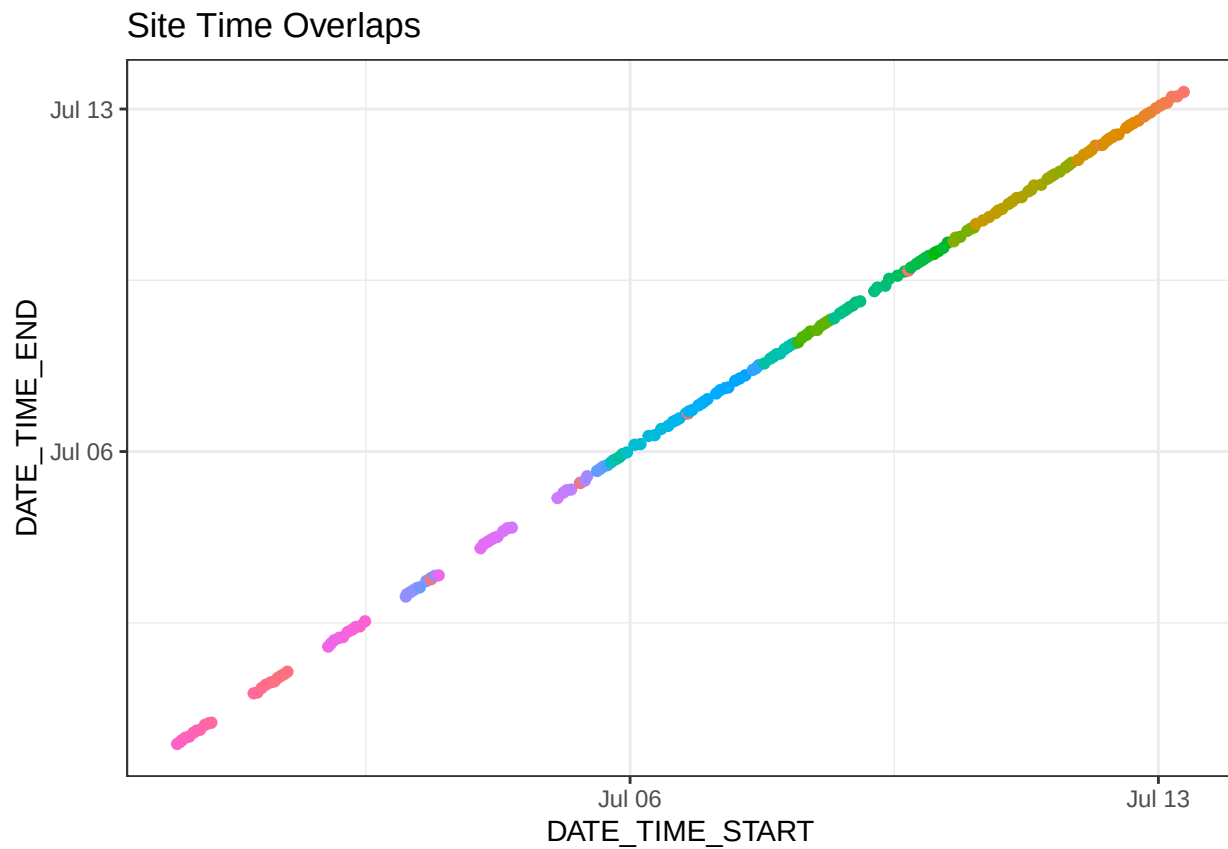
ggplot(df_plot, aes(x = `VOLUME FILTERED (L)`, y = Flow_Rate_L_min, color =
  ↪ `INSHORE.OFFSHORE`)) +
  geom_point() +
  scale_color_manual(values = c("orchid4", "orchid1")) +
  theme_bw()
```



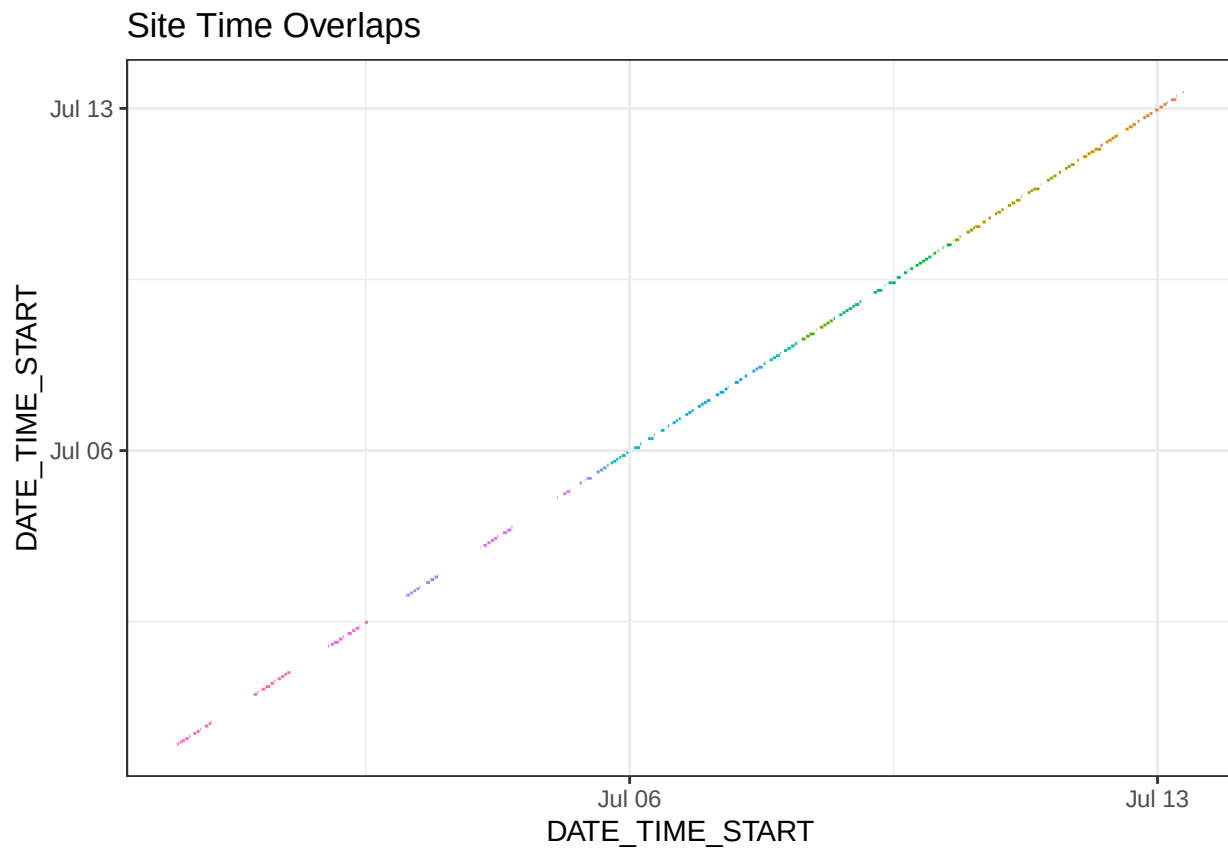
```
ggsave(filename = here("Time", "VolumeVsFlow.png"), width = 2000, height = 2000, units =
  ↪ "px")
```

```
df_plot <- df_plot %>%
  mutate(DATEFAC = as.factor(`DATE (YYYYMMDD)`))

ggplot(df_plot, aes(x = DATE_TIME_START, y = DATE_TIME_END, color = Site)) +
  geom_point() +
  theme_bw() +
  theme(legend.position="none") +
  ggtitle("Site Time Overlaps")
```

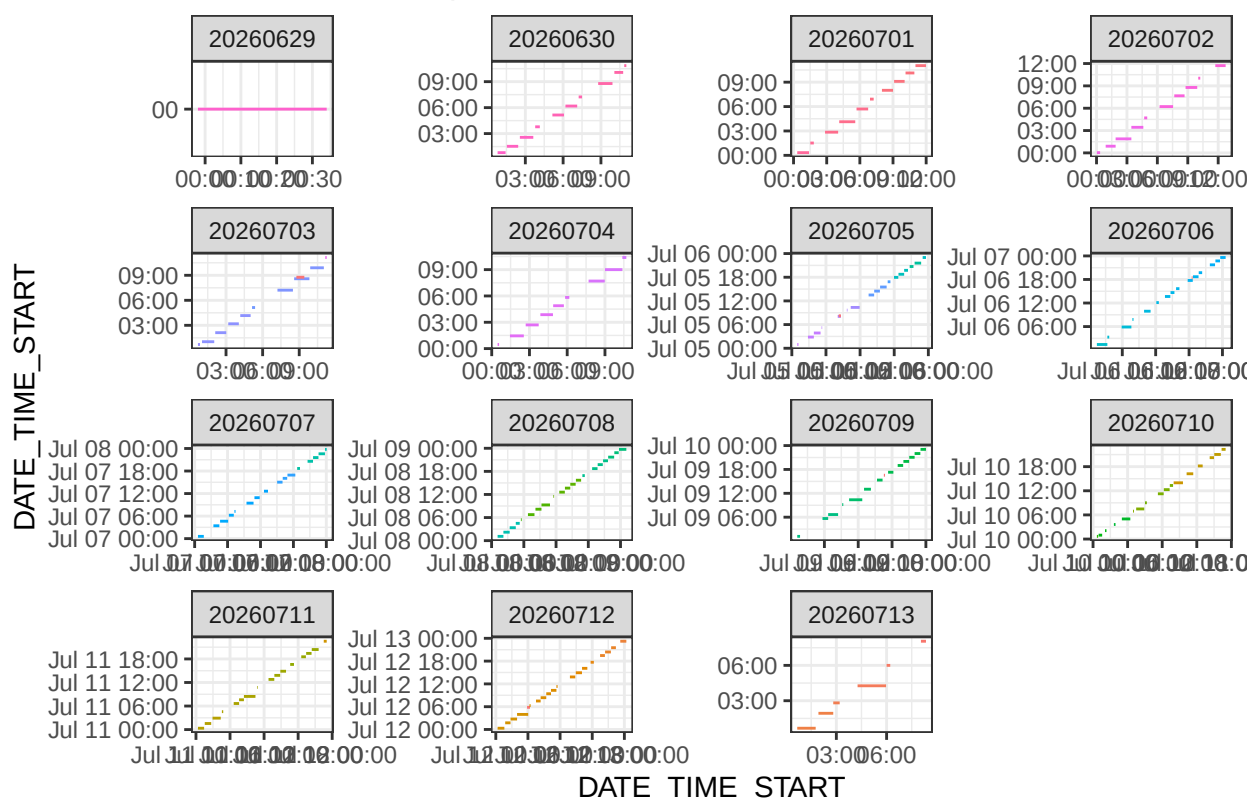


```
ggplot(df_plot, aes(x = DATE_TIME_START, xend = DATE_TIME_END, color = Site, y =
  ↳ DATE_TIME_START, yend = DATE_TIME_START)) +
  geom_segment() +
  theme_bw() +
  theme(legend.position="none") +
  ggtitle("Site Time Overlaps")
```



```
ggplot(df_plot, aes(x = DATE_TIME_START, xend = DATE_TIME_END, color = Site, y =
  ↳ DATE_TIME_START, yend = DATE_TIME_START)) +
  geom_segment() +
  theme_bw() +
  theme(legend.position="none") +
  ggtitle("Site Time Overlaps") +
  facet_wrap(facets = "DATEFAC", scales = "free")
```

## Site Time Overlaps



```
SiteOverlapExport <- ggplot(df_plot, aes(x = DATE_TIME_START, xend = DATE_TIME_END,
                                         color = `SAMPLE TYPE`,
                                         y = DATE_TIME_START, yend = DATE_TIME_START)) +
  geom_segment(linewidth = 1, lineend = "round") +
  geom_point(aes(x = DATE_TIME_START, y = DATE_TIME_START), color = "black", size = 2) +
  geom_point(aes(x = DATE_TIME_END, y = DATE_TIME_START), color = "black", size = 2) +
  theme_bw() +
  theme(legend.position="none") +
  ggtitle("Site Time Overlaps") +
  facet_wrap(facets = "DATEFAC", scales = "free", ncol = 2) +
  theme(text = element_text(size = 20))

ggsave(filename = here("Time", "Site_Sample_Breakdown.png"), plot = SiteOverlapExport,
        width = 4000, height = 6000, units = "px")
```

## RUN THIS Warning

There is one final plot that doesn't render in HTML but can be run to export plots checking that samples from the same site overlap in time reasonably. Make a folder called OverlapChecks first.

83.3-90 to 60 is weird because we skipped 80 and 70 due to weather

87.6-100 to 110 is weird, the final mosaic-b is tiny - typo! 22.35 instead of 22:35

F129 is actually a negative control

```
# not run in HTML knitting
df_plot <- df_plot %>%
```

```

mutate(PlottingID = c(1:nrow(df_plot)))

for (i in 1:length(unique(df_plot$Site))) {
  st <- unique(df_plot$Site)[i]
  print(ggplot(df_plot %>% filter(Site == st), aes(x = DATE_TIME_START, xend =
    ↪ DATE_TIME_END,
                                color = `SAMPLE TYPE`,
                                y = PlottingID, yend = PlottingID)) +
    scale_color_manual(values = c("MOSAIC-A" = "#7CAE00", "CTD" = "#F8766D",
    ↪ "MOSAIC-B" = "#00BFC4", "MOSAIC-E" = "#C77CFF", "NEG" = "#F8766D")) + # No
    ↪ CTD this time
    geom_segment(linewidth = 1, lineend = "round") +
    geom_point(aes(x = DATE_TIME_START, y = PlottingID),
               color = "black", size = 2) +
    geom_point(aes(x = DATE_TIME_END, y = PlottingID),
               color = "black", size = 2) +
    theme_bw() +
    ggtitle(paste(unique(st, "Site Time Overlaps"))) +
    theme(text = element_text(size = 20))
}

ggsave(filename = here("OverlapChecks", paste(st, "OverlapCheck.png", sep = "")), width
    ↪ = 2000, height = 2000, units = "px")
}

```